



PINAKAA Builder CI/CD

Deployment Team Onboarding & Operations Guide

Singularity/Apptainer Installation • Deployment Workflow • Container Usage • Governance



JUNE 9, 2026
C-DAC BANGALORE

1. Introduction.....	2
2. Overview of PINAKAA Builder CI/CD.....	2
3. Deployment Team Responsibilities	2
4. Access Request and Onboarding Workflow.....	3
5. Singularity / Apptainer Installation	3
5.1 Prerequisites	4
5.2 Install via Package — Recommended.....	4
Apptainer (Ubuntu/Debian via PPA)	4
Apptainer (Debian — pre-built .deb)	4
Apptainer (RHEL/Fedora via EPEL)	4
5.3 Install Unprivileged from Pre-Built Binaries	4
5.4 Install from Source (SingularityCE or Apptainer).....	5
Step 1 — Install Go	5
Step 2 — Download and Extract.....	5
Step 3 — Configure the Build	6
Step 4 — Build and Install.....	6
6. Container Deployment — Singularity/Apptainer SIF Workflow	7
6.1 Obtain the Container Image	7
6.2 Start the Container.....	7
6.3 Initialize Spack Inside the Container	7
6.4 Explore Available Packages	7
6.5 Load and Unload Packages	7
7. Deployment Validation	8
7.1 Run Package-Level Validation Scripts	8
7.2 Run a Smoke Test.....	8
8. Report Deployment Issues	8
9. Best Practices	10
10. Conclusion.....	10

1. Introduction

This document provides the official onboarding and operational workflow for the Deployment Team participating in the PINAKAA Builder CI/CD ecosystem. The Deployment Team is responsible for installing, configuring, and validating the PINAKAA software stack on client infrastructure — spanning standalone servers and computing platforms.

The guide covers:

- Singularity/Apptainer installation — via packages and from source
- Access provisioning and onboarding workflow
- Deployment of PINAKAA stack using Singularity/Apptainer containers
- Governance responsibilities as defined by the PINAKAA framework
- Best practices for reproducible, secure, and maintainable deployments

2. Overview of PINAKAA Builder CI/CD

PINAKAA Builder CI/CD provides an integrated software automation framework for onboarding, validating, building, packaging, and deploying software products across CPU, GPU, ARM, RISC-V, and accelerator-based environments. The Deployment Team is the final executor in the pipeline — receiving validated, packaged software stacks and delivering them into production HPC environments.

Deployment-relevant capabilities of the framework include:

- Singularity/Apptainer-based container readiness for reproducible, portable single-server deployments
- Validated, gatekeeper-approved packages distributed as .sif container images
- Automated build and test pipelines ensuring architecture-verified artifacts
- Heterogeneous architecture enablement: x86 (Intel/AMD), ARM (AUM), RISC-V, GPU

3. Deployment Team Responsibilities

Based on the PINAKAA Studio governance model, the Deployment Team holds the following core responsibilities:

- Deploy the final, TCG-approved software stack on client infrastructure (servers and computing platforms).
- Validate the deployment environment and confirm infrastructure readiness prior to stack installation.
- Execute deployment scripts and Singularity/Apptainer container workflows.
- Perform client environment integration and hardware-specific tuning (GPU targets, interconnects, etc.).
- Provide structured feedback on deployment issues, environment-specific failures, and compatibility gaps to Product Owners and the CI/CD Executor.

- Coordinate with the CI/CD Executor for pipeline issues and with the Application Porting Team for stack-level validation.

4. Access Request and Onboarding Workflow

Follow these steps to gain access to the PINAKAA CI/CD environment as a Deployment Team member:

1. Open the Request for Login form from the PINAKAA portal:
<https://pinakaa.cdacb.in:8448/>
2. Select the Deployment Team role during access provisioning.
3. Provide the required details:
 - Team Name and organizational email address
 - Deployment target details (server name, architecture, OS)
 - Contact and accountability information
4. Submit the onboarding request for PINAKAA administrator review and approval.
5. After approval, deployment environment credentials shall be provisioned securely.
6. Credentials and server access instructions shall be communicated through the approved official channel.

Note:

Deployment Team members receive access to validated, gatekeeper-approved packages and write access to deployment scripts and environment configuration directories only.

5. Singularity / Apptainer Installation

PINAKAA deployments use HPC-native container runtimes — SingularityCE (maintained by Sylabs) and Apptainer (maintained by the Linux Foundation). Both originate from the same Singularity codebase, use the same .sif image format, and follow an identical usage workflow. Either runtime can be used on PINAKAA deployment targets.

Compatibility Note:

.sif container images distributed via the PINAKAA portal are fully compatible with both SingularityCE and Apptainer. The container usage workflow in Section 6 is identical for both — substitute apptainer in place of singularity if Apptainer is your installed runtime.

5.1 Prerequisites

Ensure the following system packages are installed before proceeding with a source build of either runtime:

```
sudo apt-get update
sudo apt-get install -y build-essential libssl-dev uuid-dev \
    libgpgme-dev squashfs-tools libseccomp-dev wget pkg-config git cryptsetup
```

If libfuse3 is not found during the configure step, install it separately:

```
sudo apt install libfuse3-dev
```

5.2 Install via Package — Recommended

Pre-built packages are the fastest and most reliable installation method for supported Linux distributions.

Apptainer (Ubuntu/Debian via PPA)

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:apptainer/ppa
sudo apt update
sudo apt install -y apptainer
```

Verify:

```
apptainer --version
Expected output: apptainer version 1.5.0
```

Apptainer (Debian — pre-built .deb)

```
cd /tmp
wget
https://github.com/apptainer/apptainer/releases/download/v1.5.0/apptainer_1.5.0_amd64.de
b
sudo apt install -y ./apptainer_1.5.0_amd64.deb
```

Apptainer (RHEL/Fedora via EPEL)

```
sudo dnf install -y epel-release
sudo dnf install -y apptainer
```

5.3 Install Unprivileged from Pre-Built Binaries

For deployment targets without root access or package manager support, use the Apptainer unprivileged install script. This works on RHEL-derived systems, Fedora, SUSE/OpenSUSE, Debian, and Ubuntu:

```
curl -s https://raw.githubusercontent.com/apptainer/apptainer/main/tools/install-
unprivileged.sh | \
```

```
bash -s - /path/to/install-dir
```

Run using:

```
/path/to/install-dir/bin/apptainer --version
```

Note:

This script requires curl, rpm2cpio, and cpio in PATH. Multiple architectures can be installed into the same directory on a shared filesystem.

5.4 Install from Source (SingularityCE or Apptainer)

Use this method when pre-built packages are not available for the target OS. The steps below use SingularityCE v4.2.1 as the example; the procedure is the same for Apptainer source builds (refer to <https://github.com/apptainer/apptainer/blob/main/INSTALL.md> for the Apptainer source).

Step 1 — Install Go

Both runtimes require Go 1.21 or later. Download and extract the Go 1.23.4 toolchain:

```
wget https://go.dev/dl/go1.23.4.linux-amd64.tar.gz
tar -xvf go1.23.4.linux-amd64.tar.gz
export PATH=/path/to/go/bin:$PATH
```

Verify:

```
go version
Expected output: go version go1.23.4 linux/amd64
```

Tip:

Add the export PATH line to ~/.bashrc or ~/.bash_profile to persist Go across sessions.

Step 2 — Download and Extract

Download the SingularityCE v4.2.1 release tarball:

```
wget https://github.com/sylabs/singularity/releases/download/v4.2.1/singularity-ce-4.2.1.tar.gz
tar -xvf singularity-ce-4.2.1.tar.gz
cd singularity-ce-4.2.1/
```

Step 3 — Configure the Build

Run `mconfig` to configure the build. By default, the runtime installs to `/usr/local`. Use `--prefix` only if a custom installation path is required (e.g., user-space installs on HPC targets without root access):

```
./mconfig
```

Optional — install to a custom prefix:

```
./mconfig --prefix=/storage/<username>/singularity_install_dir/
```

If errors appear related to `seccomp` or `common` availability on the target system:

```
./mconfig --without-seccomp  
./mconfig --without-common
```

Note:

`--without-seccomp` disables kernel syscall filtering (acceptable in controlled HPC environments). `--without-common` disables OCI container monitoring. Use only when the system lacks these libraries.

Step 4 — Build and Install

```
cd builddir/  
make  
make install
```

If a custom prefix was used, export the binary to `PATH`:

```
export PATH=/storage/<username>/singularity_install_dir/bin:$PATH
```

Verify:

```
singularity --version  
Expected output: singularity-ce version 4.2.1
```

Tip:

Persist both Go and Singularity/Apptainer `PATH` exports in `~/.bashrc` to ensure availability across login sessions and scheduler job environments.

6. Container Deployment — Singularity/Apptainer SIF Workflow

PINAKAA packages are distributed as Singularity Image Format (.sif) container images. The workflow below applies to both SingularityCE and Apptainer — substitute apptainer in place of singularity if that is your installed runtime.

6.1 Obtain the Container Image

Download the required .sif container image from the PINAKAA portal:

```
https://pinakaa.cdacb.in:8448/
```

Verify the downloaded image is intact before use:

```
singularity inspect <container-image.sif>
```

6.2 Start the Container

Launch an interactive shell session inside the container:

```
singularity shell <container-image.sif>
```

6.3 Initialize Spack Inside the Container

Once inside the container, source the Spack environment setup script:

```
. /opt/spack/share/spack/setup-env.sh
```

6.4 Explore Available Packages

List all explicitly installed packages (top-level only, no transitive dependencies):

```
spack find -x
```

Check whether a specific package is available:

```
spack find gromacs
```

6.5 Load and Unload Packages

Load a specific package by name and version:

```
spack load gromacs@2025.3
```

Load a specific build using its unique Spack hash (preferred for reproducibility):

```
spack load /<package-hash>
```

Example:

```
spack load /7r74sn4
```


Unload all currently loaded packages:

```
spack unload --all
```

Unload a specific package build by hash:

```
spack unload /<package-hash>
```

7. Deployment Validation

After completing installation, the Deployment Team must verify that the deployed stack is functional and meets the performance requirements of the target environment.

7.1 Run Package-Level Validation Scripts

Each package includes a tests/ directory with validation scripts provided by the Product Owner. Execute these from within the deployed environment:

```
bash tests/test_script.sh
```

Review the output against the expected validation results documented in the package README.md.

7.2 Run a Smoke Test

Execute a lightweight benchmark or functional test to confirm runtime behavior:

```
# Example: GROMACS single-node smoke test
gmx mdrun -v -deffnm benchmark
```

8. Report Deployment Issues

Upload validation logs and share deployment issues with the PINAKAA team through the portal. You must be logged in to submit a deployment issue report.

Login URL:

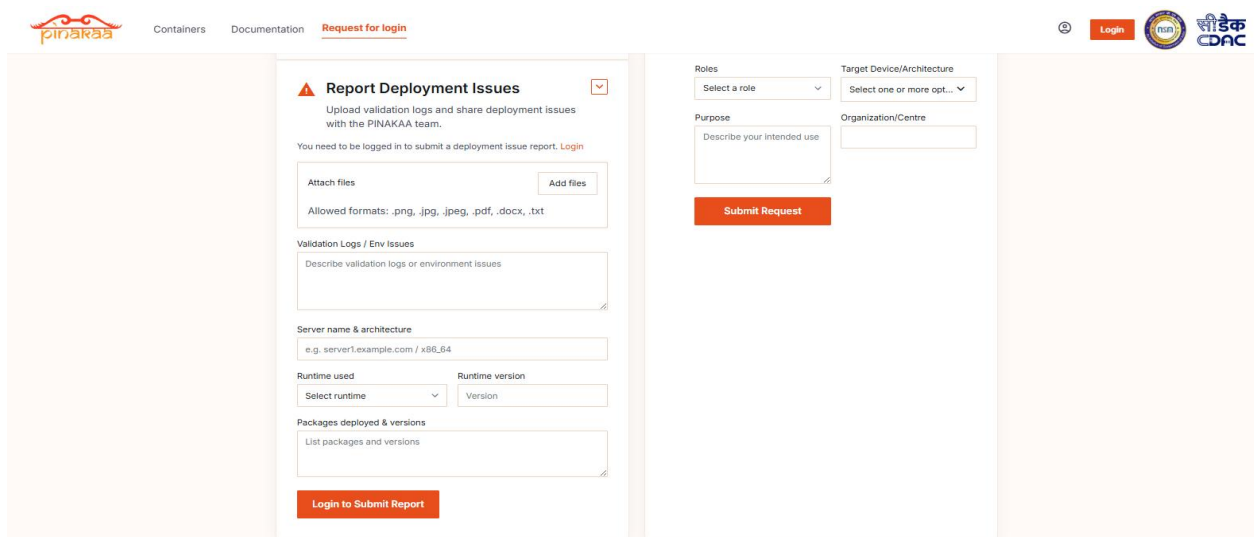
```
https://pinakaa.cdacb.in:8448/login
```

Once logged in, navigate to the Report Deployment Issues form and fill in the following details:

Field	Description
Attach Files	Add validation logs or any relevant files. Allowed formats: .png, .jpg, .jpeg, .pdf, .docx, .txt
Validation Logs / Env Issues	Describe the validation results or environment-specific issues encountered.
Server Name & Architecture	Provide the deployment target server name and its hardware architecture (e.g., x86_64, aarch64, gfx942).
Runtime Used	Select the container runtime used: • SingularityCE • Apptainer
Runtime Version	Specify the installed version of the selected runtime (e.g., singularity-ce 4.2.1 or apptainer 1.5.0).
Packages Deployed & Versions	List all packages deployed along with their versions (e.g., gromacs@2025.3, lammmps@20240829).

Note:

You must be logged in to submit the report. Use the Login link above if you are not already authenticated. Unauthenticated users will see a "Login to Submit Report" prompt on the form page.



The screenshot shows the 'Report Deployment Issues' form on the pinakaa website. The form is titled 'Report Deployment Issues' and includes a sub-header 'Upload validation logs and share deployment issues with the PINAKAA team.' Below this, a message states 'You need to be logged in to submit a deployment issue report. Login'. The form contains several input fields: 'Attach files' with an 'Add files' button, 'Validation Logs / Env Issues' with a text area, 'Server name & architecture' with a text field, 'Runtime used' with a dropdown menu, 'Runtime version' with a text field, and 'Packages deployed & versions' with a text area. A 'Login to Submit Report' button is located at the bottom of the form. To the right of the form, there are additional fields for 'Roles', 'Target Device/Architecture', 'Purpose', and 'Organization/Centre', along with a 'Submit Request' button. The top of the page features the pinakaa logo, navigation links for 'Containers', 'Documentation', and 'Request for login', and a 'Login' button next to the CDAC logo.

9. Best Practices

- Always verify the Singularity/Apptainer environment is correctly sourced before beginning any deployment activity.
- Use Spack hashes (spack load /<hash>) rather than package names alone to ensure exact, reproducible builds are loaded.
- Persist PATH exports (Go, Singularity or Apptainer) in ~/.bashrc to ensure availability across sessions and scheduler job environments.
- Run the test_script.sh provided by the Product Owner before marking a deployment as complete.
- Document all environment-specific tuning decisions (GPU targets, driver versions, runtime choice) in deployment notes.
- Raise issues through the official PINAKAA channel; do not attempt to modify Product Owner or CI/CD Executor resources directly.
- All deployment feedback must be structured: include the target system, runtime version, error logs, and reproduction steps.

10. Conclusion

The Deployment Team is the final operational link in the PINAKAA Builder CI/CD pipeline. By following the workflows in this guide — from Singularity/Apptainer installation through container-based stack deployment — the team ensures that validated, high-performance HPC and AI software reaches production environments in a reproducible, secure, and maintainable manner.

For access provisioning, deployment status updates, or issue escalation, contact the PINAKAA CI/CD coordination team: pinakaa@cdac.in